

AISHA — Feature Workflow & Operation Specification

Version 1.0 • Locked Architecture • How every major feature works from input to result

Purpose: This document explains the operational flow of the locked AISHA architecture. It describes what happens when a user interacts with each feature, which engine handles it, what data is read or written, and how the result reaches the interface/avatar. No locked module is intentionally omitted.

1. Master Operating Principle

AISHA is not a single prompt. The Language Model is one component inside a larger controlled system. The normal interaction path is:

User Action/Input → Input Processing → Identity + Mood + Bond + Memory Context → Decision Engine → Safety/Rules → Prompt Builder → Language Model → Response Validator → Response → Day Log → Encrypted Local Storage → Optional Sync → UI / Voice / Avatar

The same architecture is used for text, voice, proactive presence, reminders, mood changes, relationship updates, and many background operations, with feature-specific branches.

2. Application Interface — How the App Works

Feature	Operational flow
Home	Loads current AISHA state → retrieves safe summary of mood/bond/presence/tasks → renders avatar and controls.
Chat	Accepts user message → sends it through the AI pipeline → displays validated response → records event in Day Log.
Voice	Captures voice → speech recognition → input processing → AI pipeline → text-to-speech → lip-sync/avatar → optional encrypted voice log.
Day Logs	Requests selected day → decrypts authorized local record → builds readable timeline/summary → displays it.
Mood	Reads structured mood state → shows current values/timeline → does not expose hidden chain-of-thought.
Relationship	Reads Bond Engine state → calculates/display stage, progress and meaningful relationship events according to configured rules.
Settings	Reads/writes user preferences → validates changes → stores securely → informs affected engines/services.

3. Core Intelligence — How AISHA Decides What to Do

Identity Engine

Loads stable identity/personality/configuration → applies permanent behavioral rules → provides identity context to the Prompt Builder and Decision Engine. It does not randomly rewrite AISHA's core identity.

Mood Engine

Receives meaningful events such as conversation tone, completed tasks or configured context → updates structured short-term values (happiness, calmness, energy, affection, curiosity) within defined limits → exposes the current state to conversation, presence and avatar systems.

Love Bond Engine

Receives eligible long-term relationship events → evaluates consistency, trust, shared experiences and configured progression rules → updates bond state gradually → relationship stage can change only when defined conditions are satisfied.

Decision Engine

Combines current user input, identity, mood, bond, memory, tasks and presence context → determines the response/action category → sends the selected intent to the Safety/Rules Engine and Prompt Builder.

Presence Engine

Monitors permitted context such as app activity, return after absence and configured schedules → decides whether quiet presence, greeting, reminder or no action is appropriate → respects notification and user-control settings.

Conversation Engine

Coordinates conversation context → requests relevant memory → constructs response intent → sends context to AI integration → validates the returned response → records the final interaction.

Safety/Rules Engine

Checks boundaries, consent, privacy, prohibited behavior, user settings and product rules before an action/response is delivered. If a response fails validation, it is adjusted or regenerated.

4. AI Integration — Exact Response Pipeline

1	Input Processing	Detect language, intent and useful context.
2	Context Assembly	Load identity, current mood, bond stage, relevant memory and active task/presence context.
3	Decision	Choose response intent and interaction style.
4	Safety/Rules	Apply product rules, user boundaries and privacy constraints.
5	Prompt Builder	Build the model input from approved context.
6	Language Model	Generate natural-language candidate response.
7	Response Validator	Check consistency, tone, safety, privacy leakage and memory accuracy.
8	Delivery	Send validated response to chat or convert to voice/avatar output.
9	Logging	Record the interaction/event in the Day Log and relevant state engines.

5. Memory, Day Logs & Recall

New event: conversation/event occurs → classify importance → attach to current day → update relevant indexes/state → store securely. **Recall:** identify topic/date/context → retrieve only relevant records → decrypt/validate → provide approved context to the conversation pipeline. Missing information is not invented.

Daily finalization flow

Day container closes → conversations/events summarized → mood and bond timeline finalized → integrity check → compression where applicable → encryption → local storage → sync queue if cloud backup is enabled → next day container starts.

6. Mood & Relationship Operation

Mood: short-term state. Events can modify mood within configured bounds; mood can affect wording, presence and avatar expression. **Bond:** long-term relationship state. It changes more slowly and is based on defined relationship events rather than a single message. The bond is a software model and does not establish human consciousness or subjective emotion.

Example: meaningful conversation → Conversation Engine records event → Mood Engine may adjust short-term state → Bond Engine evaluates whether the event qualifies for long-term progression → Avatar Controller receives approved expression state → Day Log records the event.

7. Presence, Tasks & Notifications

Presence

Context/schedule trigger → Presence Engine checks user permissions and quiet settings → selects no action, quiet presence, greeting or permitted notification → UI/avatar updates.

Task Manager

User creates/edits/completes task → validation → secure local save → task status becomes available to Planner, Reminder Engine and Day Log.

Reminder Engine

Reads scheduled reminder → checks current permissions/time → schedules or triggers notification → records delivery/status.

Daily Planner

Reads user tasks/reminders and configured context → organizes daily view → user remains in control of changes.

Notification Manager

Receives permitted notification request → applies user notification settings and anti-interruption rules → schedules/delivers → logs outcome.

8. 3D Avatar — How It Reacts

Core Engine State → Avatar Controller → Expression Engine + Animation Engine → Eye/Face + Body Animation → Lip Sync when speaking → UI/live wallpaper renderer.

Expression Engine

Maps approved mood/interaction state to an expression such as neutral, happy, thoughtful or concerned.

Animation Engine

Controls breathing, blinking, posture, looking around, walking/sitting and natural idle variation.

Eye/Face

Adds natural eye contact, occasional glance-away behavior, blinking and subtle facial movement.

Lip Sync

Uses generated speech timing/audio to synchronize mouth movement.

Clothing/Appearance

Loads selected appearance independently from mood; appearance customization does not automatically change relationship state.

Live Wallpaper

Receives limited avatar state → renders ambient behavior → applies battery and user controls; it does not imply microphone/camera listening.

9. Voice System

Voice input → speech recognition → Input Processing → standard AI pipeline → validated response → text-to-speech → audio playback → lip sync/avatar → optional encrypted voice record. If voice logging is disabled, persistent voice retention is not performed by the feature.

10. Security, Authentication & Authorization

User authentication (if account mode) → authorization check → secure local access → encrypted data operations. Sensitive cloud/admin actions require server-side authorization; the client is not trusted to grant itself privileges. Security events are recorded in the audit system where appropriate.

Protected data path

Raw user data → compression where applicable → encryption → local storage and/or encrypted upload → cloud access control → retention policy.

11. Cloud, Backup & Synchronization

Local DB → encryption layer → Sync Manager → Upload Queue → cloud storage. On connectivity return, pending encrypted objects are retried. The local source is retained until successful confirmation according to the sync policy.

Download/restore: encrypted backup → integrity validation → local decryption → database/index rebuild → resume normal operation. **Conflict:** detect versions/timestamps → apply defined merge/version policy → never silently overwrite outside the defined rule.

12. Storage, Retention, Export & Trash

Storage

Working data remains locally available for normal operation. Encrypted cloud backup is optional according to account configuration.

40-day cloud retention

Cloud retention is configured at 40 days in the locked architecture. Expiry of cloud copies does not automatically delete active local data.

Export

User requests export → authorization → gather permitted records → generate readable summary and/or encrypted archive → deliver securely.

Deletion

Active record removed from normal timeline/recall → deletion state handled for sync → recovery lifecycle applied according to policy.

Protected Trash

Deleted data is encrypted/protected and excluded from normal memory recall. It is not a normal browsing source.

Recovery

Authorized recovery request → permission check → integrity validation → restore → rebuild indexes → audit event.

13. Administration & Audit

Admin login → server-side role authorization → permitted dashboard functions. Super Admin operations include higher-risk security, key-management, role and disaster-recovery workflows. Audit records cover admin logins, authorization failures, recovery, security changes, sensitive exports and relevant security events.

14. Background/System Services

Service	How it operates
Midnight/Day Finalization	Closes previous day → summarizes → validates → compresses/encrypts → queues sync → creates next day.
Sync Service	Processes upload/download queues → retries failures → verifies completion.
Notification Service	Schedules and delivers permitted notifications.
Battery Manager	Reduces background/avatar activity when device power constraints require it.
Update Manager	Checks and applies supported app/component updates while preserving data through migrations.
Crash Recovery	Restores safe state after failure and prevents corrupted sessions from silently continuing.
Storage Manager	Monitors local/cloud usage, thresholds and cleanup policies.
Connectivity Monitor	Detects network changes and starts/stops sync/retry behavior.

15. Failure Handling — What Happens When Something Breaks

No internet: Local features continue; cloud operations enter queues.

Cloud unavailable: Encrypted local data remains available; sync retries later.

Interrupted upload: Keep local source; mark object pending; retry.

AI service unavailable: Preserve state; use approved degraded/local behavior if available; inform user rather than fabricating a response.

Phone restart: Recover local database and pending queues.

App update: Run database migration and preserve compatible user data.

Phone lost: Cloud recovery is possible only when a valid backup exists; device-only data cannot be reconstructed from nothing.

Unauthorized admin: Reject operation and record security event.

Corrupt backup: Validate before restore; reject unsafe/corrupt restore rather than blindly overwriting local data.

16. Complete Example — User Sends a Normal Message

1. User types message → 2. Chat UI sends it to Input Processing → 3. Intent/language/context detected → 4. Identity Engine supplies stable personality → 5. Mood Engine supplies current short-term state → 6. Bond Engine supplies relationship stage → 7. Recall Engine retrieves relevant memories → 8. Decision Engine selects response intent → 9. Safety/Rules checks boundaries → 10. Prompt Builder creates model context → 11. Language Model generates response → 12. Response Validator checks it → 13. Final reply appears → 14. Mood/Bond engines process eligible events → 15. Avatar reacts → 16. Day Log records event → 17. Encrypted local storage saves it → 18. Sync Queue uploads later if enabled.

17. Complete Example — Voice Conversation

Microphone input (with permission) → speech recognition → same intelligence pipeline → validated text response → TTS → audio playback → lip sync → avatar expression/animation → optional encrypted voice log → Day Log → sync if enabled.

18. Complete Example — User Returns After an Absence

App opens → Presence Engine detects return → checks user settings → relevant recent context is retrieved → Decision Engine selects an appropriate greeting/presence behavior → Safety/Rules validates → response generated → Mood/avatar presentation updated → event recorded in Day Log. The system does not fabricate what happened while the user was away.

19. Feature Interaction Map

UI connects to the engines through controlled service interfaces. **Core Intelligence** coordinates identity, mood, bond, decision, presence, conversation and rules. **AI Integration** generates language only after approved context is assembled. **Memory** supplies historical context and records events. **Avatar/Voice** render approved state. **Tasks/Notifications** provide productivity behavior. **Security/Cloud/Storage** protect and persist data. **Background Services** keep the system operating across time, connectivity and app lifecycle changes.

20. Build Rule — Nothing Is Skipped

This workflow document is the operational companion to the locked module/feature specification. During implementation, each module must have: (1) defined input, (2) defined processing logic, (3) defined output/state changes, (4) storage requirements where applicable, (5) error handling, (6) security/authorization requirements where applicable, and (7) integration tests. Post-launch upgrade hooks remain separate from the initial locked implementation.

Important: the visible “thinking/activity” feature should show safe operational summaries such as “checking recent memory” or “adjusting response tone,” not private hidden chain-of-thought.